

SDI 2200284 Αλέξανδρος Γκιάφης

Εδώ είναι το folder του project μου:

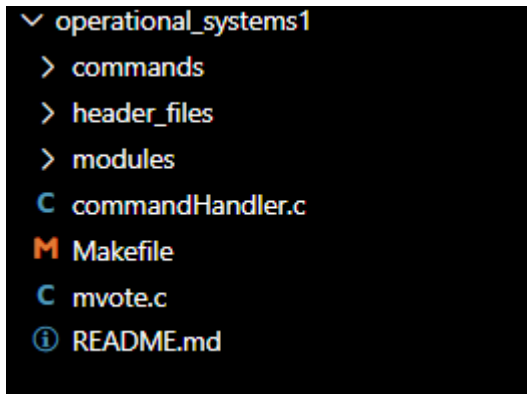


```

▼ operational_systems1
  ▼ commands
    ▼ header_files
      C command.h
      C command_bv.c
      C command_clear.c
      C command_exit.c
      C command_help.c
      C command_i.c
      C command_l.c
      C command_m.c
      C command_o.c
      C command_perc.c
      C command_save.c
      C command_z.c
      C commands_v.c
      C shared_functions.c
    ▼ header_files
      C colors.h
      C commandHandler.h
    ▼ modules
      ▼ header_files
        C hash_table.h
        C ListOfLists.h
        C voter.h
      C hash_table.c
      C ListOfLists.c
      C voter.c
      C commandHandler.c
      M Makefile
      C mvote.c
      ⓘ README.md

```

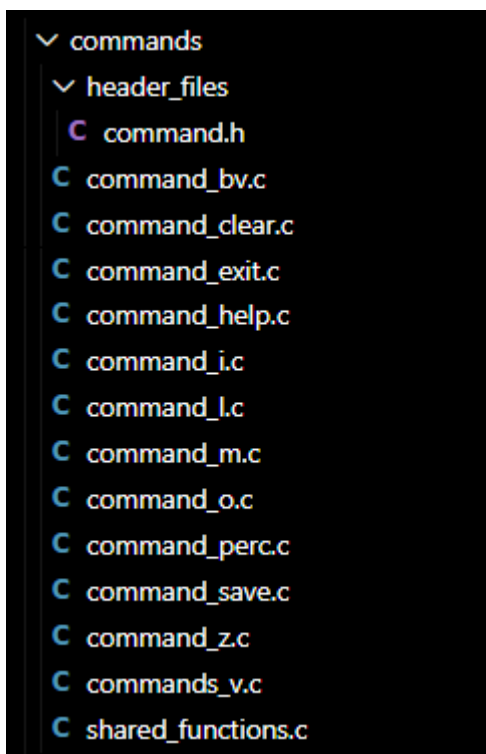
Όπως φαίνεται έχω κάνει μια προσπάθεια να οργανώσω το πρόγραμμα σε πολλά αρχεία.



Το main αρχείο είναι το mvote.c

Από εκεί διαβάζω αρχικά τα arguments του προγράμματος. Κάνω initialize τους participants στο hashtable, του οποίου η υλοποίηση βρίσκεται στα modules folder. Και μετά διαβάζω από την γραμμή εντολών γραμμές που πληκτρολογεί ο χρήστης, και τις στέλνω στον commandHandler ο οποίος τις αποδικοποιεί και καλεί το κατάλληλο function, του οποίου η υλοποίηση είναι γραμμένη στο commands folder, για την εκτέλεση της εντολής.

Στο commands folder υπάρχουν οι υλοποιήσεις των commands:



Αν θέλω να προσθέσω κάποιο άλλο command το μόνο που έχω να κάνω είναι να φτιάξω ένα νέο command file, να γράψω την υλοποίηση του function, και να κάνω update αυτό το μέρος του commandHandler.c

```

7
8 // If I ever add more commands I should modify the code here so that they are added to the array
9
10 int command_l(int* hasVotedCounter, HashTable, ListOfLists, char* delimiter);
11 int command_i(int* hasVotedCounter, HashTable, ListOfLists, char* delimiter);
12 int command_m(int* hasVotedCounter, HashTable, ListOfLists, char* delimiter);
13 int command_bv(int* hasVotedCounter, HashTable, ListOfLists, char* delimiter);
14 int command_v(int* hasVotedCounter, HashTable, ListOfLists, char* delimiter);
15 int command_perc(int* hasVotedCounter, HashTable, ListOfLists, char* delimiter);
16 int command_z(int* hasVotedCounter, HashTable, ListOfLists, char* delimiter);
17 int command_o(int* hasVotedCounter, HashTable, ListOfLists, char* delimiter);
18 int command_save(int* hasVotedCounter, HashTable, ListOfLists, char* delimiter);
19 int command_exit(int* hasVotedCounter, HashTable, ListOfLists, char* delimiter);
20 int command_clear(int* hasVotedCounter, HashTable, ListOfLists, char* delimiter);
21 int command_help(int* hasVotedCounter, HashTable, ListOfLists, char* delimiter);
22
23 typedef int (*commandFunction)(int* hasVotedCounter, HashTable, ListOfLists, char* delimiter);
24
25 struct Command {
26     const char *name;
27     commandFunction cmdFunction;
28 };
29
30 #define NUM_OF_COMMANDS 12
31
32 struct Command commands[NUM_OF_COMMANDS] = {
33     {"l", command_l},
34     {"i", command_i},
35     {"m", command_m},
36     {"bv", command_bv},
37     {"v", command_v},
38     {"perc", command_perc},
39     {"z", command_z},
40     {"o", command_o},
41     {"save", command_save},
42     {"exit", command_exit},
43     {"clear", command_clear},
44     {"help", command_help}
45 };

```

To make file θα αναλάβει να κάνει link το implementation του function από το commands folder, αρκεί να χρησιμοποιείς ίδιο function signature. Δεν χρειάζεται να αλλάξεις κάτι στο makefile, είναι έτσι φτιαγμένο ώστε να κάνει αυτόματα link τα αρχεία μέσα στο command folder.

Για παράδειγμα, ας δούμε πως έχω υλοποιήσει ένα command:

```
C command_lc X
operational_systems1 > commands > C command_lc > ...
1  #include "../header_files/command.h"
2
3  int command_l(int*, HashTable myHashTable, ListOfLists, char* delimiters){
4      // l <pin> - Prints the data of the voter of specified pin
5
6      char* parameter = strtok(NULL, delimiters);
7      if(testNumberParameter(parameter) == 0){
8          printf(ERROR_COLOR);
9          printf("Malformed Pin\n");
10         printf(RESET_COLOR);
11         return 0;
12     }
13
14     int pin = atoi(parameter);
15     Voter voter = readHashTable(myHashTable, pin, PINtoInt);
16
17     if(voter == nullItem){
18         printf(ERROR_COLOR);
19         printf("Participant %d not in cohort\n", pin);
20         printf(RESET_COLOR);
21     } else{
22         printf("%d %s %s %d\n", getPin(voter), getSurName(voter), getFirstName(voter), getTK(voter));
23     }
24 }
25
26
27
```

Χρησιμοποιώ την συνάρτηση strtok για να κάνω tokenize τις λέξεις του command που έστειλε ο χρήστης. σαν delimiters χρησιμοποιώ white spaces, αλλά επειδή αυτό μπορεί και να αλλάξει στο μέλλον ανάλογα την εφαρμογή αποφάσισα να περνάω σαν parameter τα delimiters που χρησιμοποιώ. Επίσης κάθε command έχει access σε άλλες 3 μεταβλητές, το hashTable, το ListOfLists και την μεταβλητή που μετράμε πόσοι έχουν ψηφίσει.

Νομίζω είπα αρκετά για το main πρόγραμμα, ας δούμε και τα modules τα οποία έχω φτιάξει:

```
▼ modules
  ▼ header_files
    C hash_table.h
    C ListOfLists.h
    C voter.h
  C hash_table.c
  C ListOfLists.c
  C voter.c
```

Προσπάθησα να τα φτιάξω όσο πιο generalized μπορούσα, δηλαδή να βλέπουν item και key, να μην τους ενδιαφέρει το αν το item σου είναι voter, να μην έχουν άμεση πρόσβαση στην δομή του voter, να μπορώ στο μέλλον αν θέλω, να τα πάρω και να τα χρησιμοποιήσω για ένα οποιοδήποτε άλλο πρόγραμμα, απλά αλλάζοντας κάποια defines στο header file.

Ακόμα, προσπάθησα να τα φτιάξω ώστε να χειρίζονται malloc fails, να μην αφήνουν leaks, και να διορθώνουν ότι χρειαστεί σε τέτοιες περιπτώσεις.

Για παράδειγμα, το hash table, ήταν και αυτό στο οποίο είχα το μεγαλύτερο πρόβλημα, το να διαχειρίζομαι περιπτώσεις όπου παίρνω malloc fail κατά την διάρκεια του split. Δεν ήθελα ούτε να σταματάω το πρόγραμμα του χρήστη, αλλά ούτε να αφήνω το hash table χαλασμένο. Ώστε να μπορεί μετά ο χρήστης αν θέλει να κάνει save και exit και να μην χάσει ότι πρόοδο είχε κάνει μέχρι στιγμής. Αυτό είχε ως αποτέλεσμα να αυξηθεί ελάχιστα η πολυπλοκότητα χρόνου του insert. Τίποτα υπερβολικό όμως, απλά σε περιπτώσεις insert, αν γίνει split, τότε κατά την διαδικασία του rehash των items αποθηκεύω σε μια λίστα τις αλλαγές που κάνω. Επίσης, αν όλα πάνε καλά, διαπερνώ τα buckets και μια δεύτερη φορά για να τα κάνω destroy. Αυτό διότι δεν μπορούσα να τα κάνω destroy πριν είμαι σίγουρος πως δεν θα υπάρξει πρόβλημα, γιατί πες ότι τα έκανα destroy κατά την διαδικασία του rehash, και πήγαινε να γίνει insert ένα item σε ένα άλλο index και έπρεπε να φτιάξουμε ένα νέο overflow bucket, αλλά δεν είχαμε χώρο, τότε θα έπρεπε να κάνω abort το insert, και να φέρω όλα τα items πίσω στην αρχική τους θέση, αλλά αν είχα κάνει destroy τα buckets τότε δεν θα είχα που να τα βάλω, θα έπρεπε να φτιάξω νέα, αλλά δεν θα είχα μνήμη, και άρα δεν θα μπορούσα να τα φέρω πίσω. Τέλος πάντων, για αυτό τον λόγο τα buckets τα διαγράφω μόνο εφόσον όλα πάνε καλά. Επίσης χειρίζομαι κάπως περίεργα το tail και τα numOfItems για περιπτώσεις που τα items γίνονται reinsert στο ίδιο index, ένας αλγόριθμος ο οποίος ίσως δεν είναι τελείως προφανές σε κάποιον που δεν έχει κάτσει να το σκεφτεί λίγο, αλλά θεωρητικά αν τον έχω σκεφτεί καλά πρέπει να δουλεύει, και τα tests που έχω κάνει δεν έχουν δείξει κάποιο πρόβλημα.

Το HashTable λοιπόν προφανώς θα έχει πολυπλοκότητα χρόνου $O(1)$, εφόσον όσο και αν μεγαλώσει το πλήθος των items αυτό δεν επηρεάζει τον χρόνο να γίνει insert η search. Δηλαδή, μπορεί να υπάρχουν περισσότερα overflow buckets ναι, αλλά είτε έχουμε 500 είτε 5000, περίπου τον ίδιο χρόνο θα κάνει, το hash table μεγαλώνει και γίνονται split οπότε πάντα θα υπάρχει μία ισορροπία στα overflow buckets.

ListOfLists ήταν ένα άλλο module το οποίο μου χρησίμευσε για να αποθηκεύω participants συγκεκριμένου ταχυδρομικού κώδικα όταν ψηφίσουν. Κάνω order τους ταχυδρομικούς κώδικες κατά την διαδικασία του insert. Επίσης χειρίζομαι malloc fails. Δεν ήταν τίποτα δύσκολο το συγκεκριμένο module, έχει μια πολυπλοκότητα insert $O(m)$ όπου m είναι ο αριθμός των ταχυδρομικών κωδικών, αυτό διότι κρατάω pointer στο tail των λιστών από ψηφοφόρους. Επίσης υπάρχουν κάποιες read συναρτήσεις, το ποιοι έχουν ψηφίσει σε κάποιο TK για παράδειγμα, αυτές έχουν πολυπλοκότητα $O(m * n)$ όπου n είναι ο μέσος αριθμός ατόμων που έχει το κάθε TK. Ενώ άλλες συναρτήσεις όπως αυτή που ψάχνει απλά για κάποιο TK και επιστρέφει την λίστα με τα άτομα έχει απλά $O(m)$, μετά μπορείς με πολυπλοκότητα $O(1)$ από την λίστα που επέστρεψε να πάρεις αριθμό ψηφοφόρων, και αυτό γιατί έχω listHandler το οποίο κρατάει τέτοιες πληροφορίες και άρα δεν χρειάζεται να κάτσει να τους μετρήσει αυτή την στιγμή.

Το voter είναι επίσης ένα απλό module το οποίο κάνει abstract το entity ενός Voter, δεν είναι κάτι το ιδιαίτερο.

Πως θα τρέξεις το πρόγραμμα λοιπόν. Απλά θα τρέξεις make στο terminal και το makefile που έχω θα φτιάξει το εκτελέσιμο αρχείο mvote. Όλα τα άλλα object files του προγράμματος θα τα αποθηκεύσει στον obj folder, και θα γίνεται χρήση separate compilation, οπότε μπορώ να αλλάζω πράγματα σε αρχεία και δεν θα χρειάζεται να κάνει compile ξανά τα πάντα, απλά θα κάνει compile ότι άλλαξε και μετά θα τα κάνει όλα link.

Το εκτελέσιμο πρόγραμμα θα εμφανιστεί στο ίδιο directory με το makefile, με όνομα mvote, και το χρησιμοποιείς όπως και στις οδηγίες, με τα flags από τις οδηγίες. Αν δεν θες flags τότε έχω default τιμές. Ενώ αν δεν βάλεις κάποιο file flag απλά θα αρχίζεις με ένα άδειο hash table.

Τι commands υποστηρίζει το πρόγραμμα μου? Έχω προσθέσει μερικά επιπλέον commands και μπορείς απλά να πληκτρολογήσεις help για να δεις instructions

```
Enter your command: help
This program is used to update or access the state of participants in an election.
Here is a list of all the available commands you could use:

l <PIN>                - Prints data of participant with specified PIN
i <PIN> <lname> <fname> <zip> - Adds a new participant into the program
m <pin>                - Updates the state of the participant of specified PIN to has voted
bv <filepath>          - Updates the state of the participants of specified PINs written in <filepath> to has voted
v                      - Returns number of participants that have voted so far
perc                  - Returns percentage of participants that have voted so far
z <zipcode>            - Returns the PIN of participants who have voted of specified zip code
o                      - Returns in order how many voters have voted of each zip code, order is from most voters to least voters
save <filepath>        - Saves the current state of the program in <filepath>, file can then be used with -f flag to resume another time
clear                 - Clears the terminal
help                  - Prints a list of all available commands
exit                  - Exits the program, returns how many bytes were freed in the process

Enter your command:
```